CS 5151 / 6051
Database Theory (DBT)

07: Data Analytics

Seokki Lee

Slides are adapted from:

Database System Concepts, 6th & 7th Ed. ©Silberschatz, Korth and Sudarshan

Outline

- **Overview**
- Data Warehousing
- Online Analytical Processing

Overview

- **Data analytics**: the processing of data to infer patterns, correlations, or models for prediction
- Primarily used to make business decisions
  - Per individual customer
    - E.g., what product to suggest for purchase
  - Across all customers
    - E.g., what products to manufacture/stock, in what quantity
- Critical for businesses today
- The sources may store the data under different schemas.
  - For performance reasons, the data sources usually will not permit other parts of the company to retrieve data on demand.

Overview (Cont.)

- Common steps in data analytics
  - Gather data from multiple sources into one location
    - Data warehouses also integrated data into common schema
  - **ETL (extract-transform-load)**
    - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
    - Collecting, cleaning, deduplicating, and loading

Overview (Cont.)

- Common steps in data analytics
  - Generate aggregates and reports summarizing data
    - Dashboards showing graphical charts/reports (visualization)
      - Summarizing key organizational parameters
      - Highlight anomalies
      - Provide insights into causes for changes
    - **Online analytical processing (OLAP) systems** allow interactive querying
      - Aggregate queries on large data are answered in almost real time.

Overview (Cont.)

- **Statistical analysis** using tools such as R/SAS/SPSS
  - R is widely used for statistical analysis and **graphical** display.
    - Integrated with databases as well as big data systems
  - Including extensions for parallel processing of big data
    - E.g., SparkR

Overview (Cont.)

- **Predictive models** are widely used today
  - Build predictive models and use the models for decision making
  - E.g., use customer profile features (e.g. income, age, gender, education, employment) and past history of a customer to predict likelihood of default on loan
    - And use prediction to make loan decision
  - E.g., use past history of sales (by season) to predict future sales
    - And use it to decide what/how much to produce/stock
    - And to target customers
- Other examples of business decisions:
  - What items to stock?
  - What insurance premium to change?
  - To whom to send advertisements?

Overview (Cont.)

- **Machine learning** (ML) techniques are key to finding patterns in data and making predictions
- **Data mining** extends techniques developed by machine-learning communities to run them on very large datasets
  - Knowledge discovery from ML
  - Efficient implementation techniques over large databases
- The term **business intelligence (BI)** is synonym for data analytics.
- The term **decision support** focuses on reporting and aggregation.
  - Typically use SQL queries over large amounts of data

Outline

- Overview
- **Data Warehousing**
- Online Analytical Processing

Data Warehousing

- Different data may appear
  - in different locations
  - on different operational systems
  - under different schema

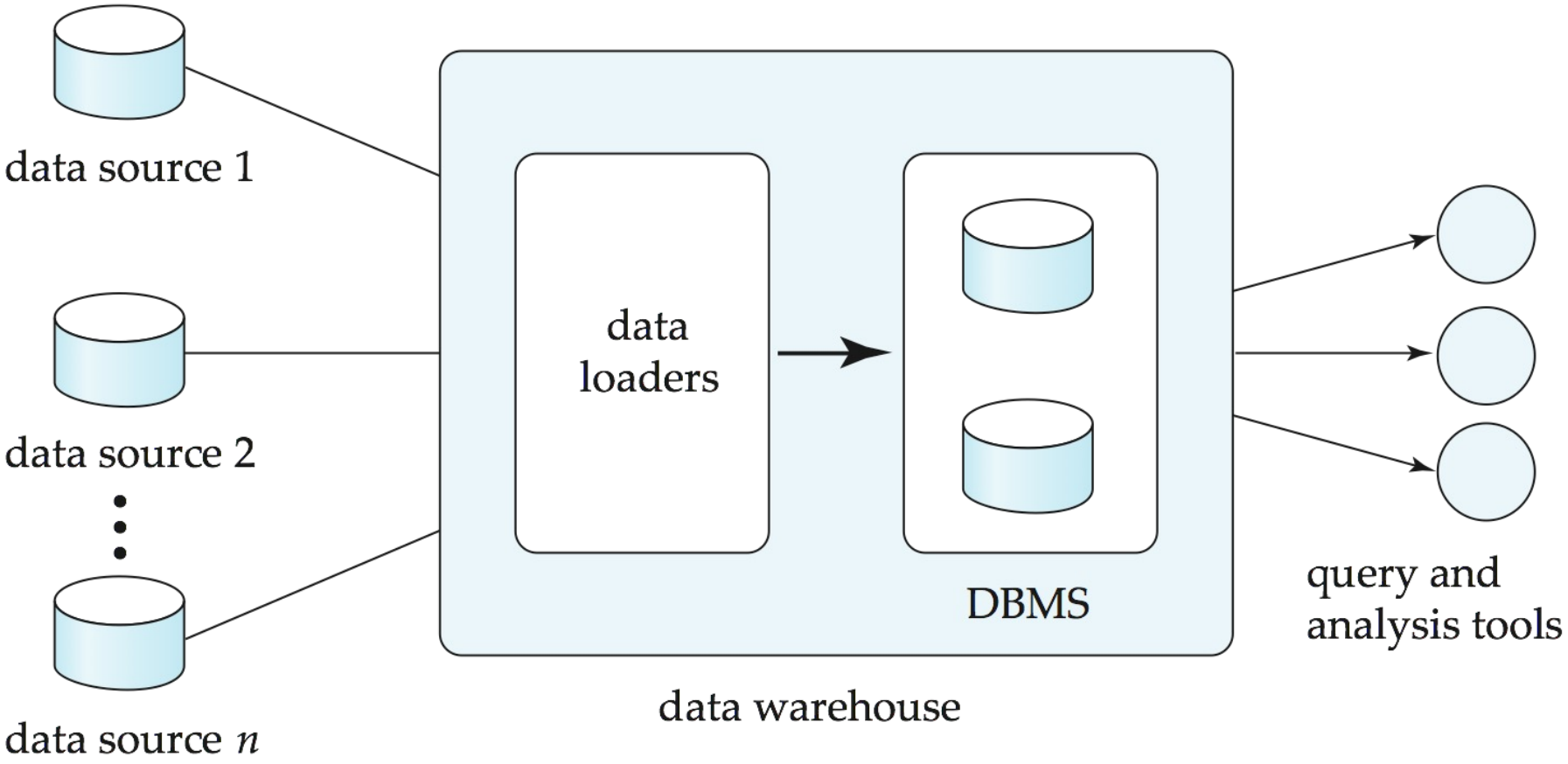
Data Warehousing

- Different data may appear
  - in different locations
  - on different operational systems
  - under different schema
- Data sources often store only current data, not historical data
  - E.g., purchase patterns
  - Corporate decision making requires a unified view of all organizational data, including historical data

Data Warehousing

- A **data warehouse** is a repository (archive) of information gathered from multiple sources, stored under a unified schema, at a single site
  - Greatly simplifies querying, permits study of historical trends
  - Shifts decision support query load away from transaction processing systems

Data Warehousing



Design Issues

- Issues to be addressed when building a data warehouse
  - When and how to gather data
  - What schema to use
  - Data transformation and cleansing
  - How to propagate updates
  - What data to summarize

Multidimensional Data and Warehouse Schemas

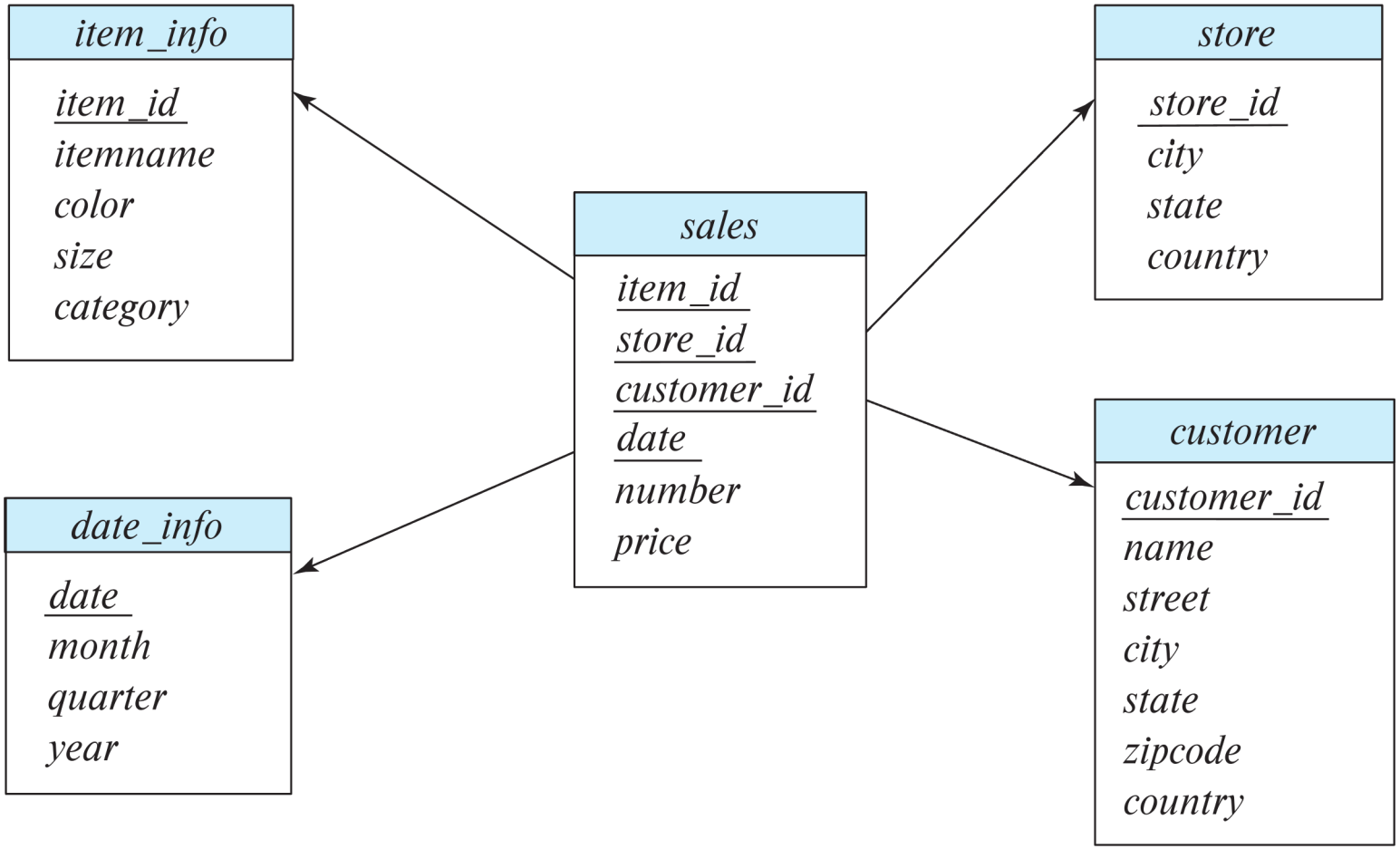
- Data warehouses typically have schemas that are designed for data analysis, using tools such as OLAP tools

- Data in warehouses can usually be divided into
  - **Fact tables**, which are usually very large
    - E.g, *sales*(*item_id, store_id, customer_id, date, number, price*)
  - **Dimension tables**, which are relatively small
    - Store extra information about stores, items, etc.

Multidimensional Data and Warehouse Schemas

- Attributes of fact tables can be usually viewed as
  - **Measure attributes**
    - measure some value, and can be **aggregated upon**
    - e.g., the attributes *number* or *price* of the *sales* relation
  - **Dimension attributes**
    - dimensions on which measure attributes are **grouped** and **viewed**
    - e.g., attributes *item_id, color,* and *size* of the *sales* relation
    - Usually small ids that are foreign keys to dimension tables

Example Data Warehouse Schema

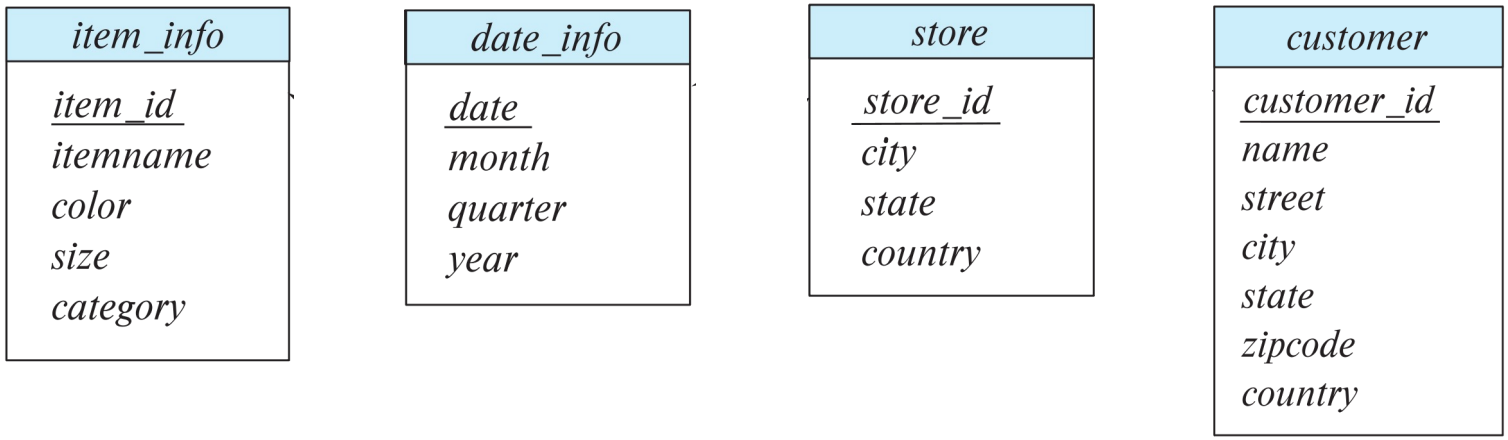


Multidimensional Data and Warehouse Schemas

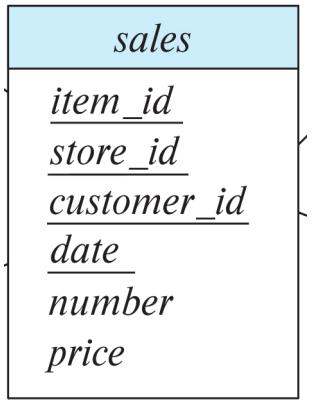
- Resultant schema in the previous slide is called a **star schema**
- More complicated schema structures
  - **Snowflake schema**: multiple levels of dimension tables
  - May have multiple fact tables

Star Schema

- Dimension tables



- Fact tables
  - Foreign key to each dimension table
  - The primary key is the combination of all foreign keys



Star Schema

- Dimension tables have redundancy
  - Values for higher levels are repeated
- Fact table is in 3NF
- Primary keys for dimension tables are typically generated
  - By using integers for better performance

Snowflake Schema

- Dimension tables may have **additional attribute(s)**
  - i.e., foreign key(s) to the immediate parent(s).
- Fact tables form the same as star schema.
- Avoiding redundancy
- Results in much more joins during query processing
- Possible to find a compromise between snowflake and star schema
  - Using snowflake for very fine-granular dimensions with many levels

Multidimensional Data and Warehouse Schemas

- Typically
  - Fact table **joined** with dimension tables
  - **Group-by** on dimension table attributes
  - **Aggregation** on measure attributes of fact table
- Some applications do not find it worthwhile to bring data to a common schema
  - **Data lakes** are repositories which allow data to be stored in multiple formats, without schema integration
  - Less upfront effort, but more effort during querying

Database Support for Data Warehouses

- Data warehouses often use **column-oriented storage**
  - E.g., a sequence of *sales* tuples is stored as follows
    - Values of item_id attribute are stored as an array
    - Values of store_id attribute are stored as an array,
  - Benefits?

Database Support for Data Warehouses

- Data warehouses often use **column-oriented storage**
  - E.g., a sequence of *sales* tuples is stored as follows
    - Values of item_id attribute are stored as an array
    - Values of store_id attribute are stored as an array,
  - Benefits?
    - Queries can fetch only attributes that they care about, reducing IO and memory cost
    - Arrays are compressed, reducing storage, IO and memory costs significantly

Database Support for Data Warehouses

- Data warehouses often use **column-oriented storage**
  - E.g., a sequence of *sales* tuples is stored as follows
    - Values of item_id attribute are stored as an array
    - Values of store_id attribute are stored as an array,
  - Benefits?
    - Queries can fetch only attributes that they care about, reducing IO and memory cost
    - Arrays are compressed, reducing storage, IO and memory costs significantly
- Data warehouses often use parallel storage and query processing infrastructure
  - Distributed file systems, Map-Reduce, Hive, …

Outline

- Overview
- Data Warehousing
- **Online Analytical Processing**

Data Analysis and OLAP

- **Online Analytical Processing (OLAP)**
  - Interactive analysis of data, allowing data to be summarized and viewed in different ways in an online fashion (with negligible delay)
- We use the following relation to illustrate OLAP concepts
  - *sales* (*item_name*, *color*, *clothes_size*, *quantity*)

  This is a simplified version of the *sales* fact table joined with the dimension tables, and many attributes removed (and some renamed)

Example Sales Relation

item_name	*color*	*clothes_size*	*quantity*
dress	dark	small	2
dress	dark	medium	6
dress	dark	large	12
dress	pastel	small	4
dress	pastel	medium	3
dress	pastel	large	3
dress	white	small	2
dress	white	medium	3
dress	white	large	0
pants	dark	small	14
pants	dark	medium	6
pants	dark	large	0
pants	pastel	small	1
pants	pastel	medium	0
pants	pastel	large	1
pants	white	small	3
pants	white	medium	0
pants	white	large	2
shirt	dark	small	2
shirt	dark	medium	6
shirt	dark	large	6
shirt	pastel	small	4
shirt	pastel	medium	1
shirt	pastel	large	2
shirt	white	small	17
shirt	white	medium	1
shirt	white	large	10
skirt	dark	small	2
skirt	dark	medium	5
...	...	...	...
...	...	...	...

Cross Tabulation of *sales* by *item_name* and *color*

clothes_size **all**

item_name \ *color*	dark	pastel	white	total
skirt	8	35	10	53
dress	20	10	5	35
shirt	14	7	28	49
pants	20	2	5	27
total	62	54	48	164

Cross Tabulation of *sales* by *item_name* and *color*

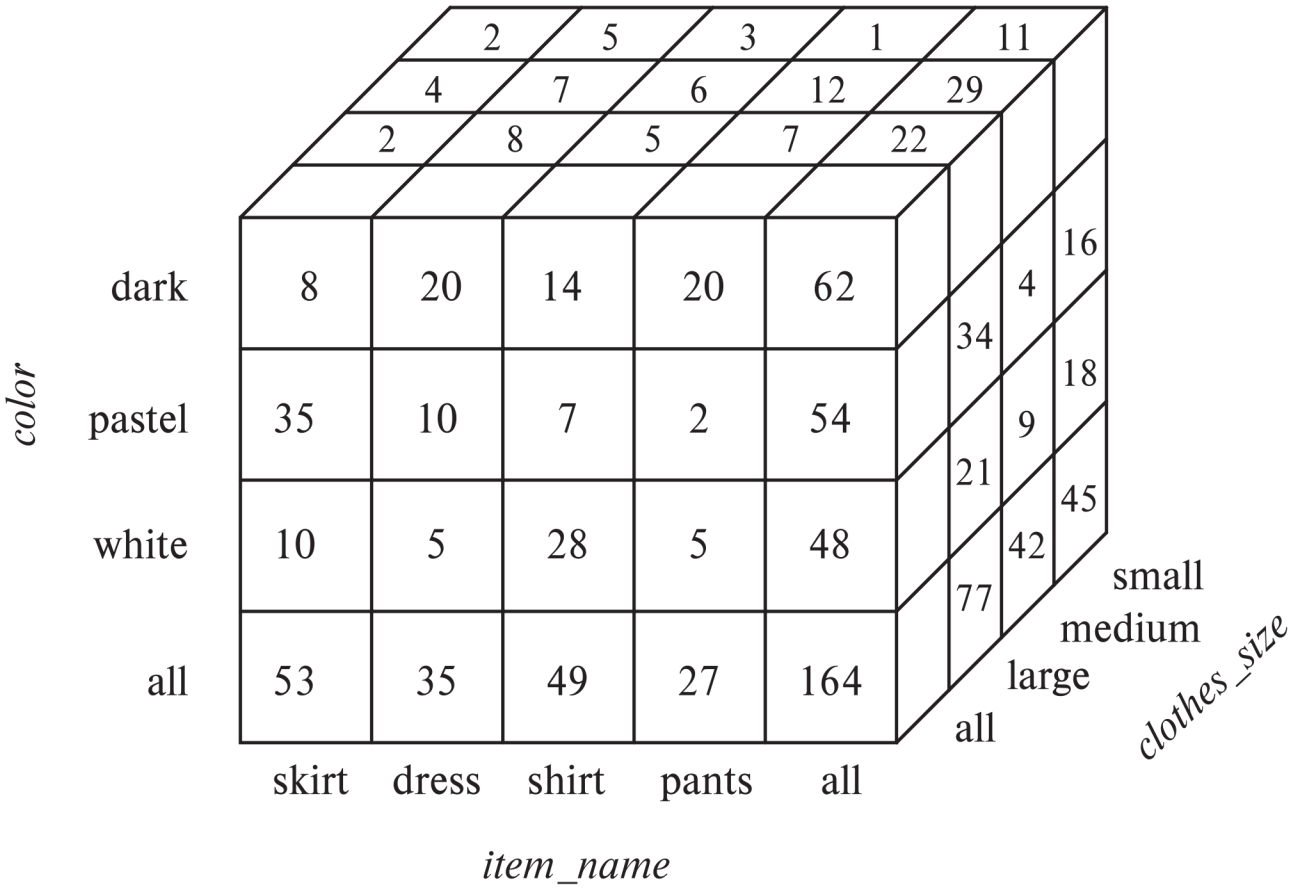
clothes_size **all**

item_name	*color*: dark	pastel	white	total
skirt	8	35	10	53
dress	20	10	5	35
shirt	14	7	28	49
pants	20	2	5	27
total	62	54	48	164

- The table above is an example of a **cross-tabulation** (**cross-tab**).
  - Referred to as a **pivot-table**
  - Values for one of the dimension attributes form the row headers
  - Values for another dimension attribute form the column headers
  - Other dimension attributes are listed on top
  - Values in individual cells are (aggregates of) the values of the dimension attributes that specify the cell.

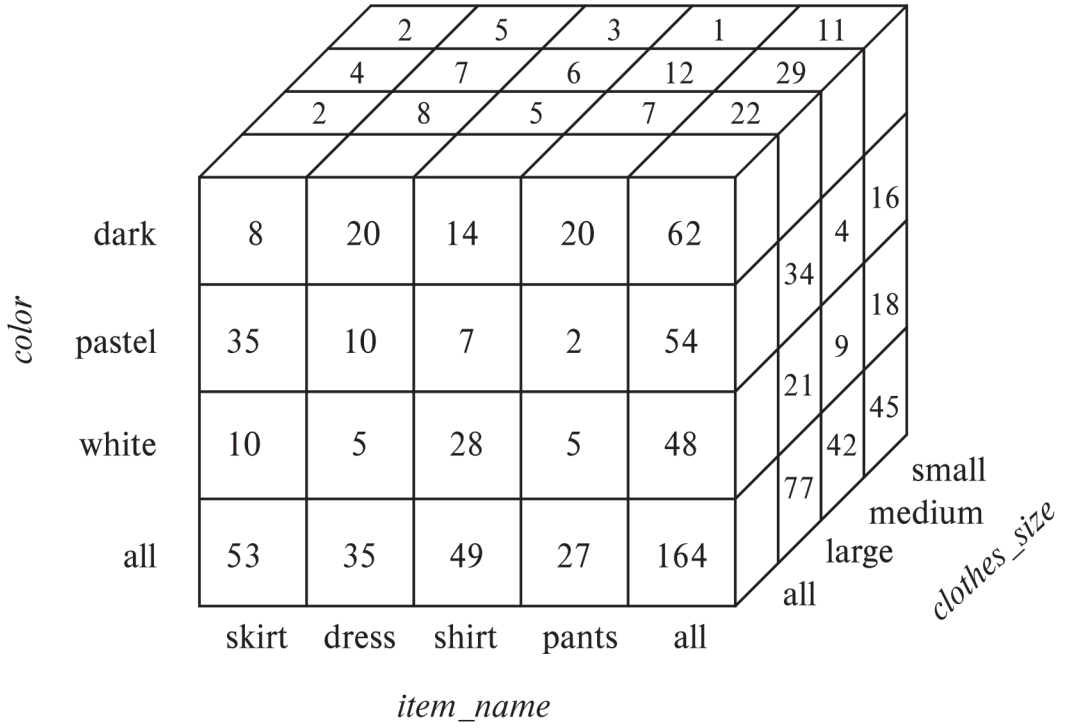
Data Cube

- A **data cube** is a multidimensional generalization of a cross-tab
- Can have n dimensions; we show 3 below
- Cross-tabs can be used as views on a data cube.



Data Cube

- Each **cell** corresponds to a combination of elements from each dimension.
  - **Facts** are non-empty cells
  - Cells store **measures**
- Cube for a combination of levels of the dimension

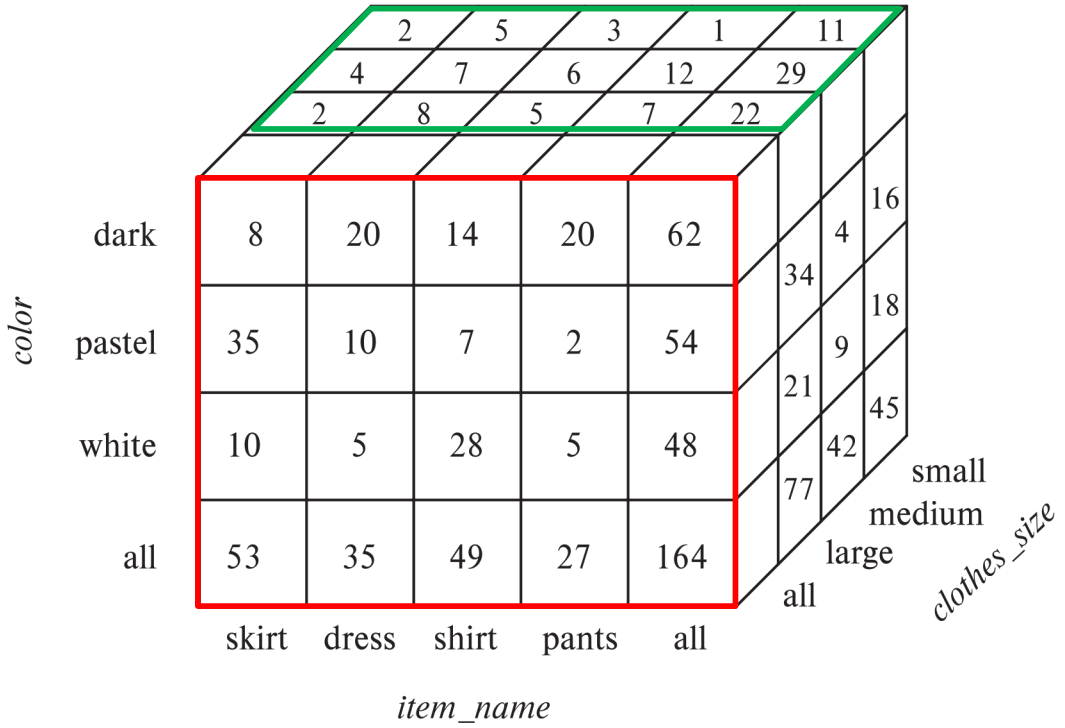


Online Analytical Processing Operations

- Pivoting
- Slicing
- Rollup
- Drill down

Online Analytical Processing Operations

- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names

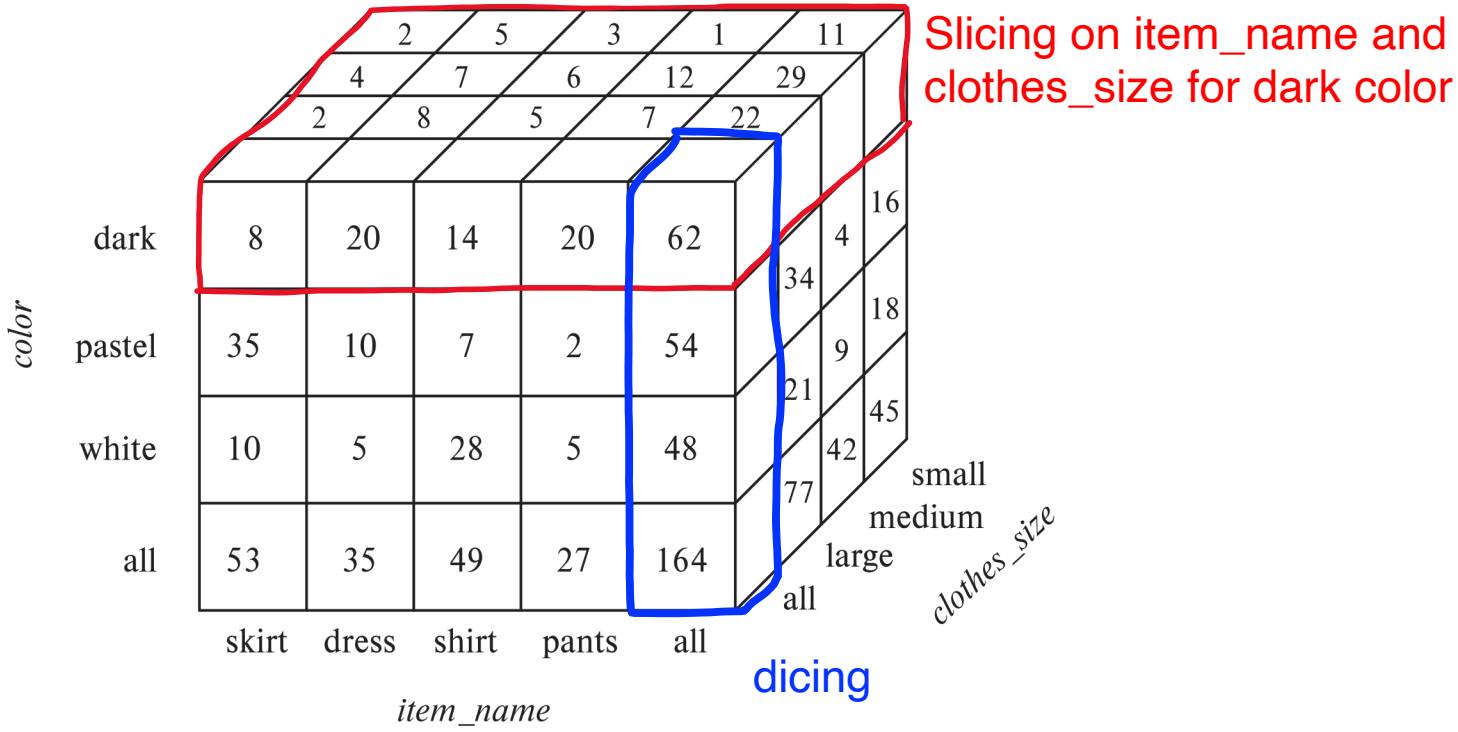


Online Analytical Processing Operations

- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing**

Online Analytical Processing Operations

- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
  - E.g., fixing color to white and size to small
  - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.



Online Analytical Processing Operations

- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
  - E.g., fixing color to white and size to small
  - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.
- **Rollup**

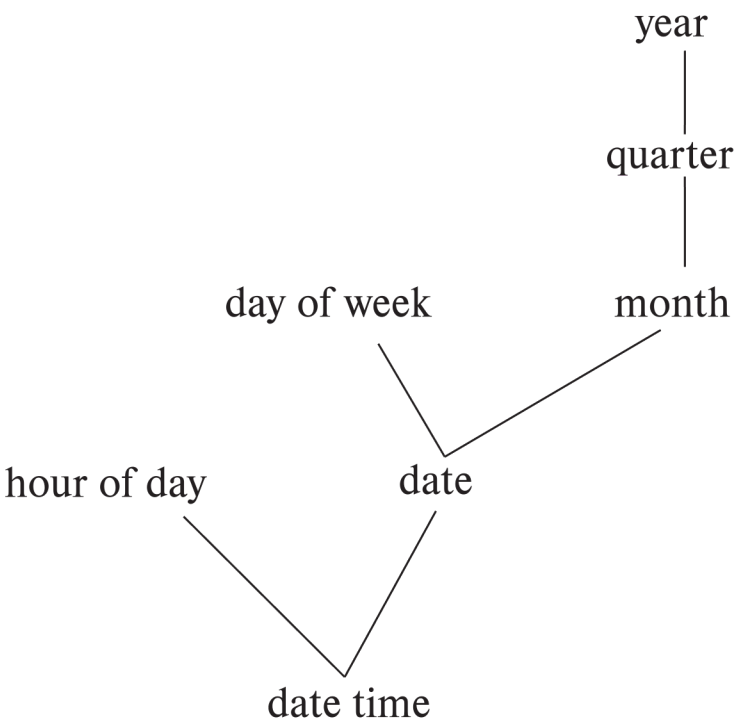
- **Drill down**

Online Analytical Processing Operations

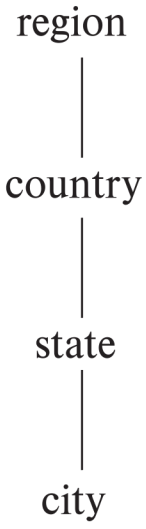
- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
  - E.g., fixing color to white and size to small
  - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.
- **Rollup:** moving from finer-granularity data to a coarser granularity
  - E.g., aggregating away an attribute
  - E.g., moving from aggregates by day to aggregates by month or year
- **Drill down:** The opposite operation - that of moving from coarser-granularity data to finer-granularity data

Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
  - E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year



(a) time hierarchy



(b) location hierarchy

Cross Tabulation With Hierarchy

- Cross-tabs can be easily extended to deal with hierarchies.
- Can drill down or roll up on a hierarchy
  - E.g. hierarchy: *item_name* → *category*

clothes_size: **all**

category	*item_name*	*color*				
		dark	pastel	white	total	
womenswear	skirt	8	8	10	53	
	dress	20	20	5	35	
	subtotal	28	28	15		88
menswear	pants	14	14	28	49	
	shirt	20	20	5	27	
	subtotal	34	34	33		76
total		62	62	48		164

Pivot in SQL

- **select** *
  from *sales*
  pivot (
  sum(*quantity*)
  for *color* **in** ('dark','pastel','white')
)
  order by *item_name*;
- Specifying how values are combined
- Not computing the subtotals

item_name	*clothes_size*	*dark*	*pastel*	*white*
dress	small	2	4	2
dress	medium	6	3	3
dress	large	12	3	0
pants	small	14	1	3
pants	medium	6	0	0
pants	large	0	1	2
shirt	small	2	4	17
shirt	medium	6	1	1
shirt	large	6	2	10
skirt	small	2	11	2
skirt	medium	5	9	5
skirt	large	1	15	3

Pivot in SQL

- **Pivot** operation in Postgres
  - *Tablefunc* module
    - **create extension** tablefunc;
  - **select** *
    from crosstab (
    ‘ -- sql statement -- ‘
)
    as ct(row_names, category_1, … category_n**)**;

Pivot in SQL

- **Pivot** operation in Postgres
  - **select** *
    from crosstab (
    'select item_name, clothes_size, color, sum(quantity)
    from sales
    group by item_name, clothes_size, color
    order by 1,2,3'
)
    as ct(item_name text,
    clothes_size text,
    dark int,
    pastel int,
    white int) ;

item_name	*clothes_size*	*dark*	*pastel*	*white*
dress	small	2	4	2
dress	medium	6	3	3
dress	large	12	3	0
pants	small	14	1	3
pants	medium	6	0	0
pants	large	0	1	2
shirt	small	2	4	17
shirt	medium	6	1	1
shirt	large	6	2	10
skirt	small	2	11	2
skirt	medium	5	9	5
skirt	large	1	15	3

Relational Representation of Cross-tabs

- Cross-tabs are different from the tables normally stored in a database
- We use the value **all** to represent aggregates.
- The SQL standard actually uses *null* values in place of **all**
  - Works with any data type
  - But can cause confusion with regular null values.

item_name	*color*	*clothes_size*	*quantity*
skirt	dark	**all**	8
skirt	pastel	**all**	35
skirt	white	**all**	10
skirt	**all**	**all**	53
dress	dark	**all**	20
dress	pastel	**all**	10
dress	white	**all**	5
dress	**all**	**all**	35
shirt	dark	**all**	14
shirt	pastel	**all**	7
shirt	white	**all**	28
shirt	**all**	**all**	49
pants	dark	**all**	20
pants	pastel	**all**	2
pants	white	**all**	5
pants	**all**	**all**	27
all	dark	**all**	62
all	pastel	**all**	54
all	white	**all**	48
all	**all**	**all**	164

OLAP in SQL

- The **cube** operation computes union of **group by**'s on every subset of the specified attributes
- E.g., consider the query

  select *item_name, color, clothes_size,* **sum**(*quantity*)
  from *sales*
  group by cube(*item_name, color, clothes_ size*)

OLAP in SQL

- The **cube** operation computes union of **group by**'s on every subset of the specified attributes
- E.g., consider the query

  select *item_name, color, clothes_size,* **sum**(*quantity*)
  from *sales*
  group by cube(*item_name, color, clothes_ size*)

  This computes the union of eight different groupings of the *sales* relation:

  { (*item_name, color, size*), (*item_name, color*),
  (*item_name, size*), (*color, size*),
  (*item_name*), (*color*),
  (*size*), ( ) }

  where ( ) denotes an empty **group by** list.
- For each grouping, the result contains the null value for attributes not present in the grouping.

OLAP in SQL

- Relational representation of cross-tab that we saw earlier, but with ***null*** in place of **all**, can be computed by

```
select *
from (select item_name, color, clothes_ size, sum(quantity)
        from sales
        group by cube(item_name, color, clothes_ size))
where clothes_size is null;
```

item_name	*color*	*clothes_size*	*quantity*
skirt	dark	**null**	8
skirt	pastel	**null**	35
skirt	white	**null**	10
skirt	**null**	**null**	53
dress	dark	**null**	20
dress	pastel	**null**	10
dress	white	**null**	5
dress	**null**	**null**	35
shirt	dark	**null**	14
shirt	pastel	**null**	7
shirt	white	**null**	28
shirt	**null**	**null**	49
pants	dark	**null**	20
pants	pastel	**null**	2
pants	white	**null**	5
pants	**null**	**null**	27
null	dark	**null**	62
null	pastel	**null**	54
null	white	**null**	48
null	**null**	**null**	164

OLAP in SQL

- The function **grouping**() can be applied on an attribute
  - Returns 1 if the value is a null value representing all, and returns 0 in all other cases.

item_name	*color*	*clothes_size*	*quantity*
skirt	dark	**all**	8
skirt	pastel	**all**	35
skirt	white	**all**	10
skirt	**all**	**all**	53
dress	dark	**all**	20
dress	pastel	**all**	10
dress	white	**all**	5
dress	**all**	**all**	35
shirt	dark	**all**	14
shirt	pastel	**all**	7
shirt	white	**all**	28
shirt	**all**	**all**	49
pants	dark	**all**	20
pants	pastel	**all**	2
pants	white	**all**	5
pants	**all**	**all**	27
all	dark	**all**	62
all	pastel	**all**	54
all	white	**all**	48
all	**all**	**all**	164

OLAP in SQL

- The function **grouping**() can be applied on an attribute
  - Returns 1 if the value is a null value representing all, and returns 0 in all other cases.
  - **select case when grouping**(*item_name*) = 1 **then** '**all**'
    else *item_name* **end as** *item_name*,
    case when grouping(*color*) = 1 **then** '**all**'
    else *color* **end as** *color*,
    '**all**' **as** *clothes_size*, **sum**(*quantity*) **as** *quantity*
    from *sales*
    group by cube(*item name*, *color, clothes_size*);

item_name	*color*	*clothes_size*	*quantity*
skirt	dark	**all**	8
skirt	pastel	**all**	35
skirt	white	**all**	10
skirt	**all**	**all**	53
dress	dark	**all**	20
dress	pastel	**all**	10
dress	white	**all**	5
dress	**all**	**all**	35
shirt	dark	**all**	14
shirt	pastel	**all**	7
shirt	white	**all**	28
shirt	**all**	**all**	49
pants	dark	**all**	20
pants	pastel	**all**	2
pants	white	**all**	5
pants	**all**	**all**	27
all	dark	**all**	62
all	pastel	**all**	54
all	white	**all**	48
all	**all**	**all**	164

OLAP in SQL

- Can use the function **decode**() in the **select** clause to replace such nulls by a value such as **all**
  - E.g., replace *item_name* in first query by
    decode(**grouping**(item_*name*), 1, 'all', *item_name*)

Extended Aggregation

- The **rollup** construct generates union on every prefix of specified list of attributes
  - **select** *item_name*, *color*, *size*, **sum**(*number*)
    from *sales*
    group by rollup(*item_name, color, size*)

Extended Aggregation

- The **rollup** construct generates union on every prefix of specified list of attributes
  - **select** *item_name*, *color*, *size*, **sum**(*number*)
    from *sales*
    group by rollup(*item_name, color, size*)
  - Generates union of four groupings:

    { (*item_name, color, size*), (*item_name, color*), (*item_name*), ( ) }

Extended Aggregation

- The **rollup** construct generates union on every prefix of specified list of attributes
  - **select** *item_name*, *color*, *size*, **sum**(*number*)
    from *sales*
    group by rollup(*item_name, color, size*)
  - Generates union of four groupings:

    { (*item_name, color, size*), (*item_name, color*), (*item_name*), ( ) }
- Rollup can be used to generate aggregates at multiple levels of a hierarchy.
  - E.g., suppose table *itemcategory*(*item_name, category*) gives the category of each item. Then would give a hierarchical summary by *item_name* and by *category.*

    select *category, item_name*, **sum**(*number*)
    from *sales, itemcategory*
    where *sales.item_name = itemcategory.item_name*
    group by rollup(*category, item_name*)

Extended Aggregation

- Multiple rollups and cubes can be used in a single group by clause
  - Each generates set of group by lists, cross product of sets gives overall set of group by lists.
  - **select** *item_name, color, size*, **sum**(*number*)
    from *sales*
    group by rollup(*item_name*), **rollup**(*color, size*)

Extended Aggregation

- Multiple rollups and cubes can be used in a single group by clause
  - Each generates set of group by lists, cross product of sets gives overall set of group by lists
  - **select** *item_name, color, size*, **sum**(*number*)
    from *sales*
    group by rollup(*item_name*), **rollup**(*color, size*)

    {item_name, ()} X {(color, size), (color), ()}

    = { (*item_name, color, size*), (*item_name, color*), (*item_name*), (*color, size*), (*color*), ( ) }

Extended Aggregation

- Neither **rollup** nor **cube** clause gives complete control on the groupings.
  - {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}

Extended Aggregation

- Neither **rollup** nor **cube** clause gives complete control on the groupings.
  - {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}
  - **select** *item_name*, *color*, *clothes_size*, **sum**(*quantity*)
    from *sales*
    group by grouping sets ((*color*, *clothes_size*),
    (*clothes_size*, *item_name*));

OLAP Implementation

- The earliest OLAP systems used multidimensional arrays in memory to store data cubes, and are referred to as **multidimensional OLAP (MOLAP)** systems.
- OLAP implementations using only relational database features are called **relational OLAP (ROLAP)** systems
- Hybrid systems, which store some summaries in memory and store the base data and other summaries in a relational database, are called **hybrid OLAP (HOLAP)** systems.

OLAP Implementation (Cont.)

- Early OLAP systems precomputed *all* possible aggregates in order to provide online response
  - Space and time requirements for doing so can be very high
    - 2^n combinations of **group by**

OLAP Implementation (Cont.)

- Early OLAP systems precomputed *all* possible aggregates in order to provide online response
  - Space and time requirements for doing so can be very high
    - 2^n combinations of **group by**
  - It suffices to precompute some aggregates, and compute others on demand from one of the precomputed aggregates
    - Can compute aggregate on (*item_name, color*) from an aggregate on (*item_name, color, size*)
      - For all but a few "non-decomposable" aggregates such as *median* is cheaper than computing it from scratch

item_name	*color*	*clothes_size*	*quantity*
skirt	dark	**all**	8
skirt	pastel	**all**	35
skirt	white	**all**	10
skirt	**all**	**all**	53

Reporting

- **Reporting tools** help create formatted reports with tabular/graphical representation of data
  - SQL Server reporting services (Microsoft)
  - Crystal Reports (SAP)

Acme Supply Company, Inc.
Quarterly Sales Report

Period: Jan. 1 to March 31, 2009

Region	Category	Sales	Subtotal
North	Computer Hardware	1,000,000	
	Computer Software	500,000	
	All categories		1,500,000
South	Computer Hardware	200,000	
	Computer Software	400,000	
	All categories		600,000
		Total Sales	2,100,000

Data Visualization

- Graphical representation of data (going beyond the basic chart)
- Data visualization tools help create **interactive** visualization of data
  - E.g., Tableau, FusionChart, plotly, Datawrapper, Google Charts, etc.
  - Frontend typically based on HTML+JavaScript



Corresponding Reading Materials

- Database System Concepts 7th Edition
  - Chapter 11
- Database System Concepts 6th Edition
  - Chapter 20